Compiler Design

Preet Kanwal
Department of Computer Science & Engineering

Teaching Assistant : Kavya P K

Compiler Design

Unit 3: L-Attributed SDD

Preet Kanwal

Department of Computer Science & Engineering

Lecture Overview

In this lecture, you will learn about -

- What is an L-attributed SDD?
- L–Attributed SDD Examples
  - Simple Type declaration
  - Array type Variable Declaration
  - Variable declaration verification
  - Desk calculator

Recap

- Syntax Directed Definitions (SDD) are a generalization of context-free grammars in which -
  - Grammar symbols have an associated set of attributes
  - Productions are associated with Semantic Rules for computing the values of attributes.
- Two kinds of attributes
  - Synthesized Attributes - computed from the values of the attributes of the children nodes.
  - Inherited Attributes - computed from the values of the attributes of both the siblings and the parent nodes.
- An SDD with only synthesized attributes is called an S-attributed definition.

What is an L-attributed SDD?

A Syntax Directed Definition is **L-attributed** if all attributes are either -

1. Synthesized
2. Extended synthesized attributes, which can depend not only on attributes at the children, but on inherited attributes at the node itself.
3. Inherited, but depending only on inherited attributes at the parent and any attributes at siblings to the left.

L-attributed SDD

- The formal definition of an L-Attributed SDD is as follows -

  A syntax directed definition is L-Attributed if each inherited attribute of X_j in a production $A \rightarrow X_1 \ldots X_j \ldots X_n$, depends only on -

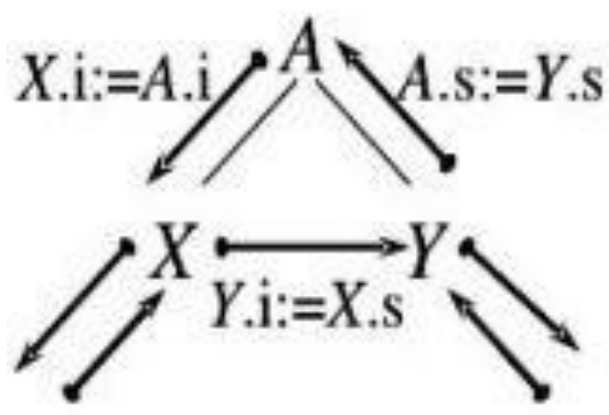
  1. The attributes of the symbols to the left (this is what L in L-Attributed stands for) of X_j, i.e., $X_1 X_2 \ldots X_{j-1}$
  2. The inherited attributes of A.

- Theorem - Inheritedattributes in L-Attributed Definitions can be computed by a PreOrder traversal of the parse-tree.

L-attributed SDD

- L-attributed definitions allow for the natural order of evaluating attributes, i.e, depth first, left to right.
- For example -

A → X Y



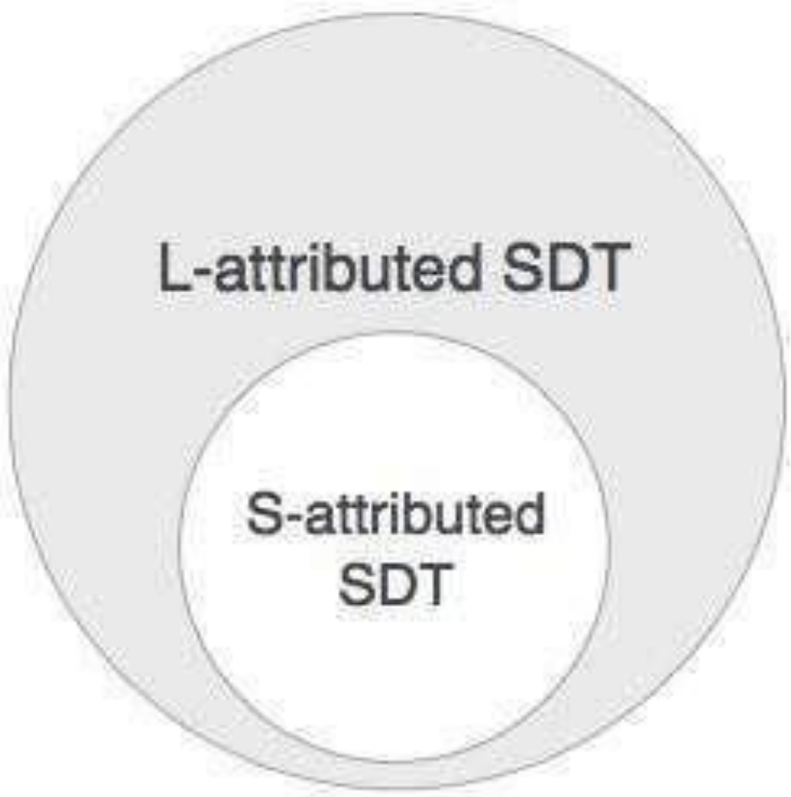
$X.i := A.i$

$Y.i := X.s$

$A.s := Y.s$

L-attributed SDD

Every S-attributed Syntax-Directed Definition is also L-attributed.



L-Attributed SDD to implement a Simple Desk Calculator

Complete the semantic rules for the following L-attributed SDD.

Evaluate the SDD for the input 3 + 5

Production	Semantic Rule
$E \rightarrow T\ E'$	
$E' \rightarrow +\ T\ E'_1$	
$E \rightarrow \lambda$	
$T \rightarrow F\ T'$	
$T' \rightarrow *\ F\ T'_1$	
$T' \rightarrow \lambda$	
$F \rightarrow num$	

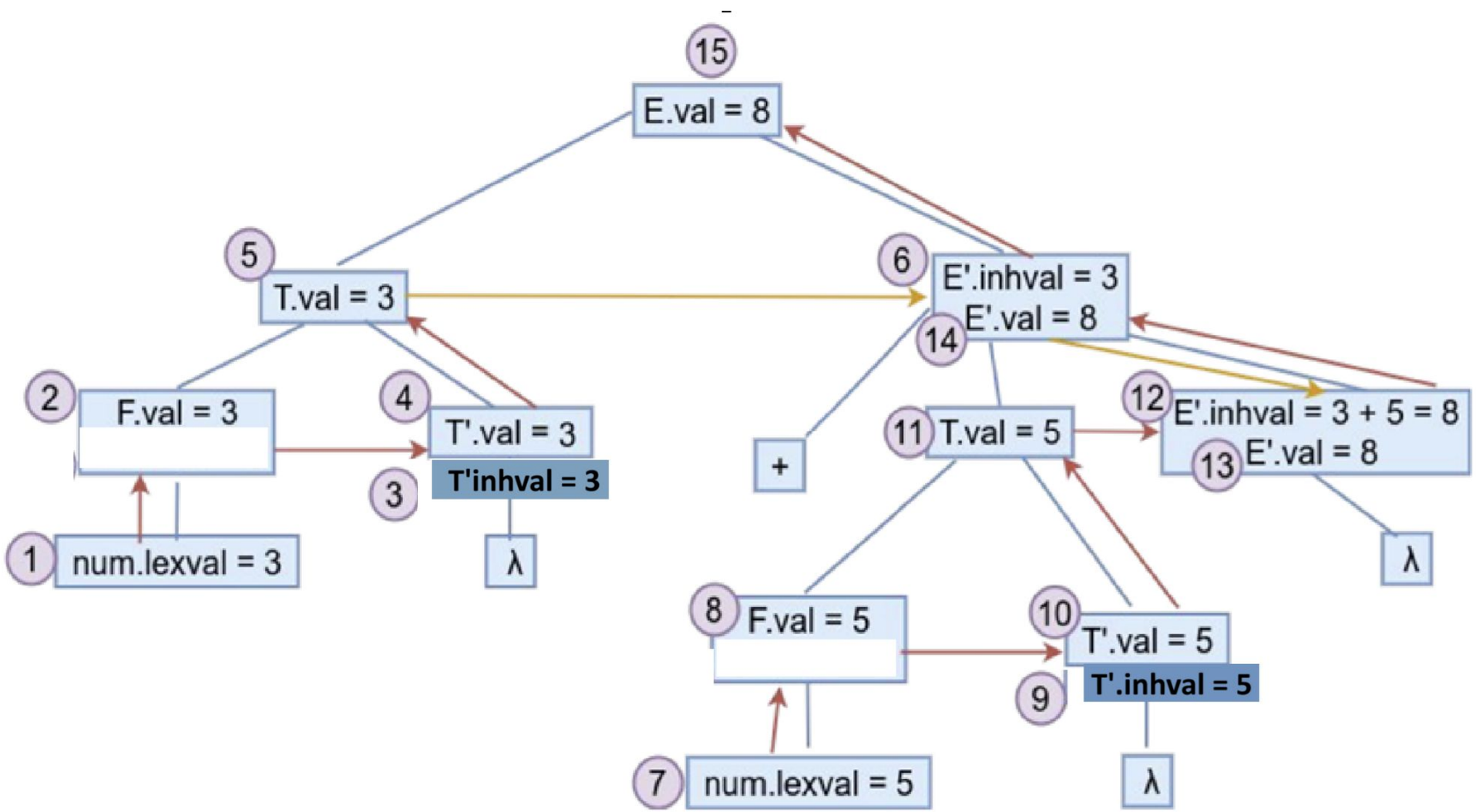
L-Attributed SDD to implement a Simple Desk Calculator

Production	Semantic Rule
$E \rightarrow T\ E'$	$\{ E'.inhval = T.val;\ E.val = E'.val; \}$
$E' \rightarrow +\ T\ E'_1$	$\{ E'_1.inhval = E'.inhval + T.val;\ E'.val = E'_1.val; \}$
$E' \rightarrow \lambda$	$\{ E'.val = E'_1.inhval; \}$
$T \rightarrow F\ T'$	$\{ T'.inhval = F.val;\ T.val = T'.val; \}$
$T' \rightarrow *\ F\ T'_1$	$\{T'_1.inhval = T'.inhval + F.val;\ T'.val = T'_1.val; \}$
$T' \rightarrow \lambda$	$\{ T'.val = T'.inhval \}$

This is an LDD - In $E \rightarrow T\ E'$, the attribute for E' is inherited from T, which is the left-sibling.

L-Attributed SDD to implement a Simple Desk Calculator

Evaluate the SDD for the input 3 + 5



L-attributed SDD for Simple Type declaration

Complete the semantic rules for the following L-attributed SDD.

Evaluate the SDD for the input int a,b

Production	Semantic Rule
$D \rightarrow T\ L$	
$T \rightarrow$ int	
$T \rightarrow$ float	
$L \rightarrow L_1$, id	
$L \rightarrow$ id	

L-attributed SDD for Simple Type declaration

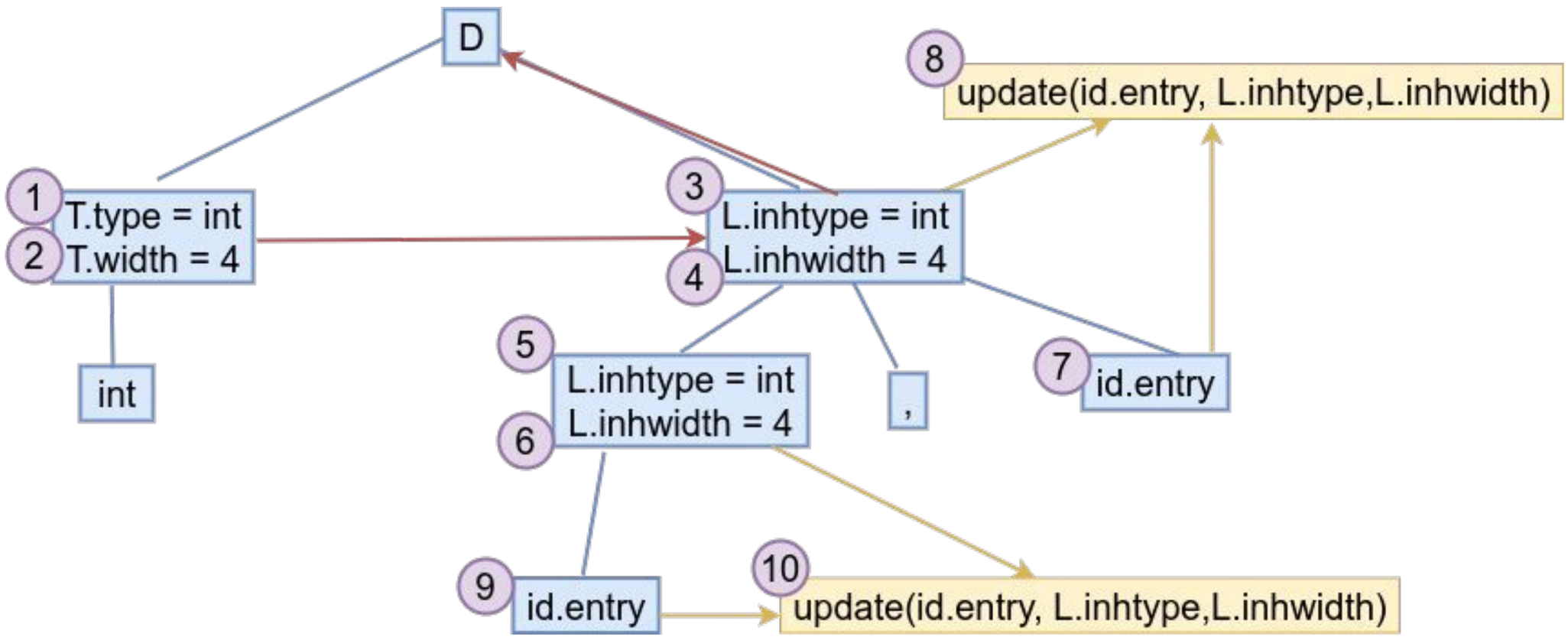
Production	Semantic Rule
$D \rightarrow T\ L$	*{ L.inhType = T.type; L.inhWidth = T.width; }*
$T \rightarrow$ int	*{ T.type = integer; T.width = 4;}*
$T \rightarrow$ float	*{ T.type = float; T.width = 8; }*
$L \rightarrow L_1$, id	*{ L_1.inhType = L.inhType; L_1.inhWidth = L.inhWidth; update(id.entry, L.inhType,L.inhWidth); }*
$L \rightarrow$ id	*{update(id.entry, L.inhType,L.inhWidth); }*

This is an LDD - In $D \rightarrow T\ L$, the attribute for L is inherited from T, which is the left-sibling.

update() is used to update type and storage in the symbol table.

L-attributed SDD for Simple Type declaration

Evaluate the SDD for the input int a,b;



L-attributed SDD to identify Array Type

Complete the semantic rules for the following L-attributed SDD.

Evaluate the SDD for the input int [2][3]

Production	Semantic Rule
$T \rightarrow B\ C$	
$B \rightarrow$ int	
$B \rightarrow$ float	
$C \rightarrow$ [num] C_1	
$C \rightarrow \lambda$	

L-attributed SDD to identify Array Type

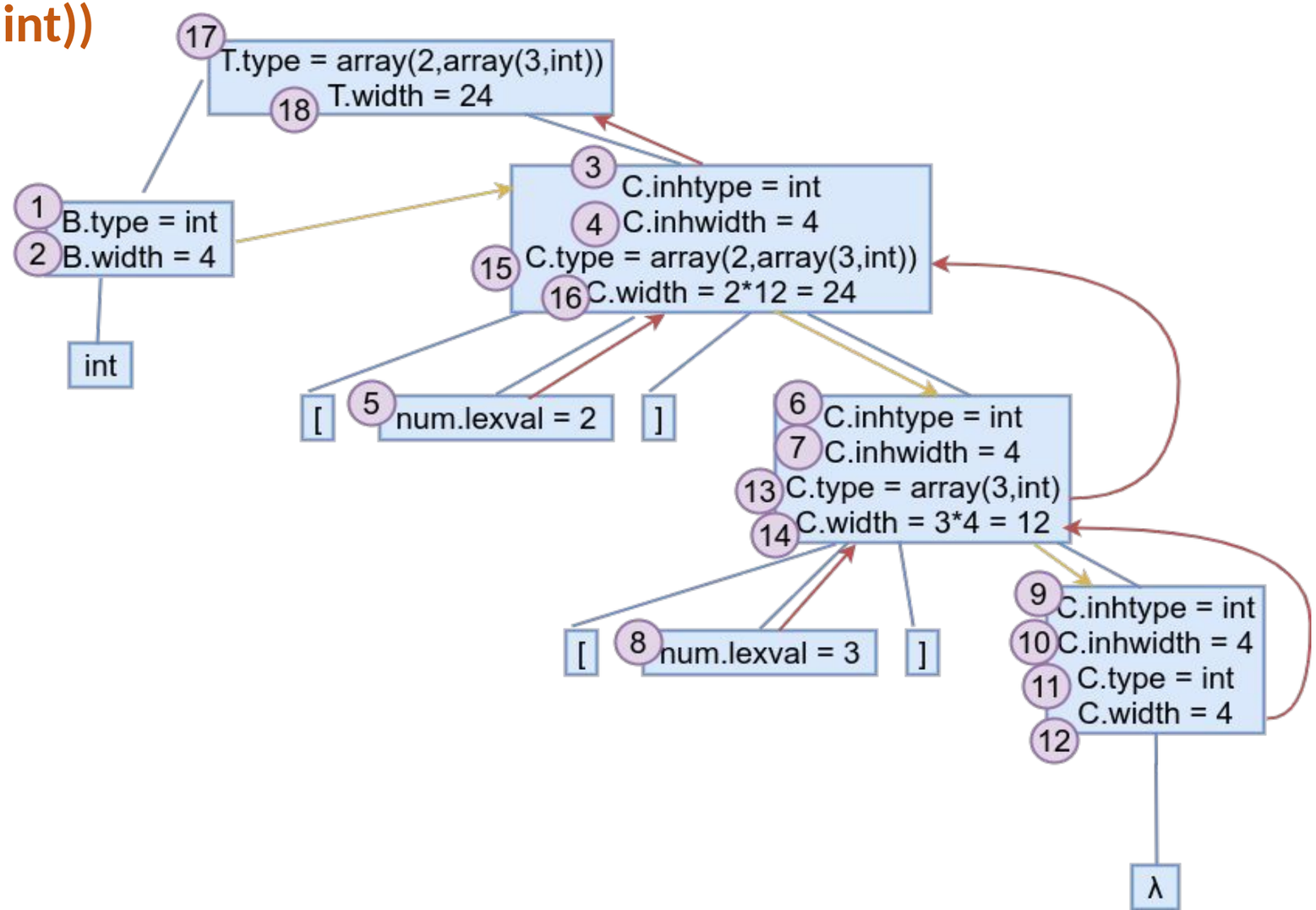
Production	Semantic Rule
$T \rightarrow B\ C$	{ C.inhType = B.type; C.inhWidth = B.width; T.type = C.type; T.width = C.width; }
$B \rightarrow$ int	{ B.type = integer;B.width = 4;}
$B \rightarrow$ float	{ B.type = float; B.width = 8; }
$C \rightarrow$ [num] C_1	{ C_1.inhType = C.inhType; C_1.inhWidth = C.inhWidth; C.type = array(num.lexval , C_1.type); C.width = num.lexval * C_1.width ;); addType(id.entry, L.inhType); addWidth(id.entry, L.inhWidth; }
$C \rightarrow \lambda$	{ C.type = C.inhType; C.width = C.inhWidth; }

L-attributed SDD for Simple Type declaration

Evaluate the SDD for the input int a[2][3]

int a[2][3] translates to

array(2,array(3,int))



L-attributed SDD for Basic vs Array Type declaration in C Semantics

Construct an SDD to identify an array of the following format - float[4] x, y

Follow C Semantics - i.e, x is an array type, y is a basic type

Production	Semantic Rule
$D \rightarrow T\ L$	
$T \rightarrow B\ C$	
$B \rightarrow int$	
$B \rightarrow float$	
$C \rightarrow [num]\ C_1$	
$C \rightarrow \lambda$	
$L \rightarrow L_1\ ,\ id$	
$L \rightarrow id$	

L-attributed SDD for Basic vs Array Type declaration in C Semantics

Production	Semantic Rule
D→ T L	{ L.inhType = T.type; L.inhWidth = T.width; L.inhbasicType = T.basicType; L.inhbasicWidth = T.basicWidth; }
T → B C	{ C.inhType = B.type; C.inhWidth = B.width; T.type = C.type; T.width = C.width; T.basicType = B.type; T.basicWidth = B.width;}
B → int	{ B.type = integer;B.width = 4;}
B→ float	{ B.type = float; B.width = 8; }
C → [num] C_1	{ C_1.inhType = C.inhType; C_1.inhWidth = C.inhWidth; C.type = array(num.lexval , C_1.type); C.width = num.lexval * C_1.width; }
C → λ	{ C_1.inhType = C.inhType; C_1.inhWidth = C.inhWidth; }

L-attributed SDD for Basic vs Array Type declaration in C Semantics

Production	Semantic Rule
…	…

$L \rightarrow L_1$, id	L_1.inhType = L.inhType;
	L_1.inhWidth = L.inhWidth;
	L_1.inhbasicType = L.inhbasicType;
	L_1.inhbasicWidth = L.inhbasicWidth;
	addType(id.entry, L.inhbasicType);
	addWidth(id.entry, L.inhbasicWidth;

L → id	{ addType(id.entry, L.inhType); addWidth(id.entry, L.inhWidth; }

THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

Compiler Design

Preet Kanwal
Department of Computer Science & Engineering

Teaching Assistant : Kavya P K

Compiler Design

Unit 3: Syntax Directed Definitions

Preet Kanwal

Department of Computer Science & Engineering

Lecture Overview

In this lecture, you will learn about -

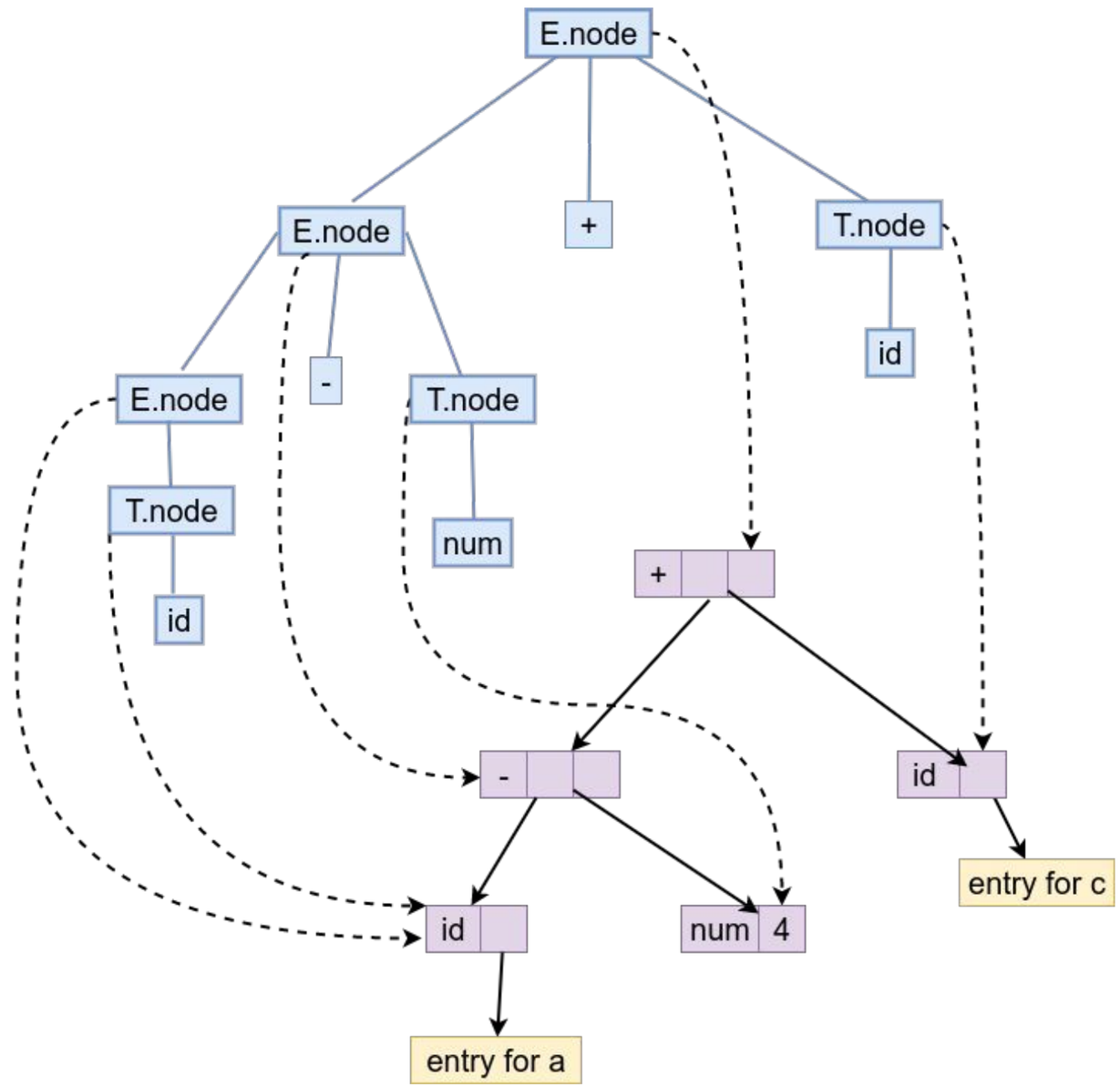
- S–Attributed SDD Examples :
  - To generate Syntax tree for Expressions
  - To generate Syntax tree for Statements

Example 1 - SDD to generate Syntax tree for Expressions

Production	Semantic Rule
E -> E_1 + T	{ E.node = new Node('+' , E_1.node, T.node); }
E -> E_1 - T	{ E.node = new Node('-' , E_1.node, T.node); }
E -> T	{ E.node = T.node ; }
T -> (E)	{ T.node = E.node ; }
T -> id	{ T.node = new Leaf(id , id.entry); }
T -> num	{ T.node = new Leaf(num , num.lexval); }

Example 1 - SDD to generate Syntax tree for Expressions

Use the previous grammar to construct the syntax tree for the input a - 4 + c



Example 2 - SDD to generate Syntax tree for Statements

Production	Semantic Rule
Stmt -> S Stmt	{ Stmt.node = new Node(Seq, S.node, Stmt.node); }
Stmt -> S	{ Stmt.node = S.node; }
S -> if (cond) { Stmt }	{ S.node = new Node(if, Cond.node, Stmt.node); }
S -> while (cond) { Stmt }	{ S.node = new Node(while, Cond.node, Stmt.node); }
S -> AssignExpr	{ S.node = AssignExpr.node ; }
Cond -> E_1 > E_2	{ Cond.node = new Node(>, E_1.node, E_2.node); }
Cond -> E_1 < E_2	{ Cond.node = new Node(<, E_1.node, E_2.node); }
Cond -> E_1 || E_2	{ Cond.node = new Node(||, E_1.node, E_2.node); }
Cond -> E_1 && E_2	{ Cond.node = new Node(&&, E_1.node, E_2.node); }
AssignExpr -> id = E;	{ AssignExpr.node = new Node(=, new Leaf(id,id.entry), E.node); }

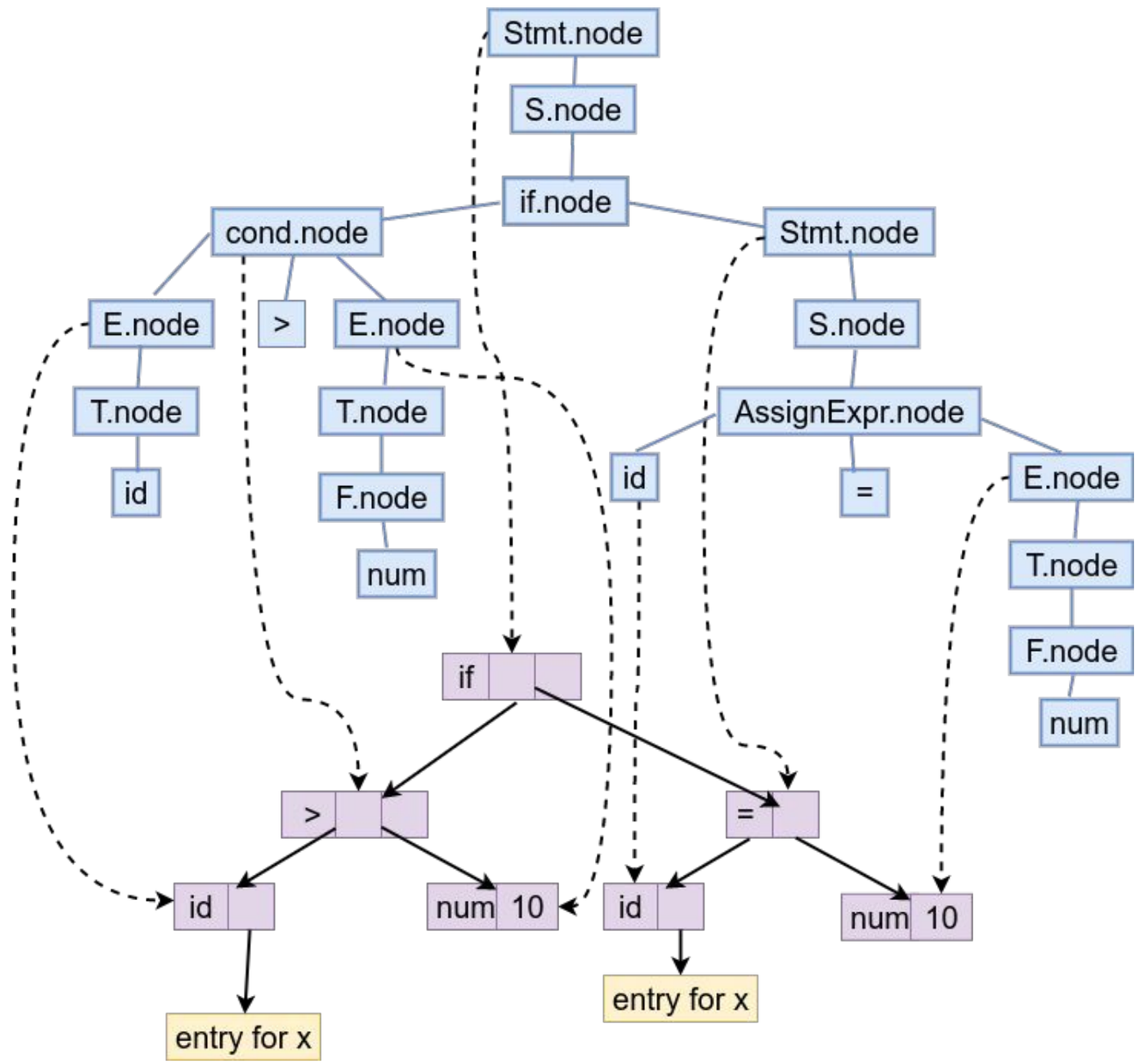
Example 2 - SDD to generate Syntax tree for Statements

Production	Semantic Rule
…	…
$E \rightarrow E_1 + T$	*{ E.node = new Node('+' , E_1.node, T.node); }*
$E \rightarrow T$	*{ E.node = T.node; }*
$T \rightarrow T_1 * F$	*{ T.node = new Node('*' , T_1.node, F.node); }*
$T \rightarrow F$	*{ T.node = F.node; }*
$F \rightarrow id$	*{ F.node = new Leaf(id , id.entry); }*
$F \rightarrow num$	*{ F.node = new Leaf(num , num.lexval); }*

Example 2 - SDD to generate Syntax tree for Statements

Use the previous grammar to construct the syntax tree for the input

```
if ( x > 10 )
{
    x = 10;
}
```



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

Compiler Design

Preet Kanwal
Department of Computer Science & Engineering

Teaching Assistant : Sree Pranavi G

Compiler Design

Unit 3: L-Attributed SDD - Intermediate Code Generation

Preet Kanwal

Department of Computer Science & Engineering

Lecture Overview

In this lecture, you will learn about -

- L–Attributed SDD to generate intermediate code for -
  - Expressions
  - Condition statement
  - If statement
  - If - else statement
  - While statement
  - Do - while statement
  - For statement
  - Boolean expressions

Recap

- There are 2 kinds of attributes - Synthesized and Inherited.
- An SDD with only synthesized attributes is an S-attributed definition.
- An SDD is L-attributed if all its attributes are either -
  - Synthesized
  - Extended synthesized dependent on children as well as inherited attributes.
  - Inherited but dependent only on inherited attributes at parent and any siblings at left.
- Every S-attributed SDD is also L-attributed.

SDD to generate Intermediate Code - Arithmetic Expressions

Write the SDD to generate intermediate code.
The given example indicates the code and its corresponding intermediate code for an expression a = b + - c .

Input	Output
a = b + - c	*t1 = minus c* *t2 = b + t1* *a = t2*

SDD to generate Intermediate Code - Arithmetic Expressions

Assigning appropriate semantic rules to each production -

Production	Semantic Rule
S -> id = E;	*S.code = E.code || gen(id.lexval '=' E.addr)*
E -> E1 + E2	*E.addr = new Temp();* *E.code = E1.code || E2.code ||* *gen(E.addr '=' E1.addr '+' E2.addr)*
E -> -E1	*E.addr = new Temp();* *E.code = E1.code || gen(E.addr '=' 'minus' E1.addr)*
E -> (E1)	*E.addr = E1.addr* *E.code = E1.code*
E -> id	*E.addr = id.lexval* *E.code = ' '*

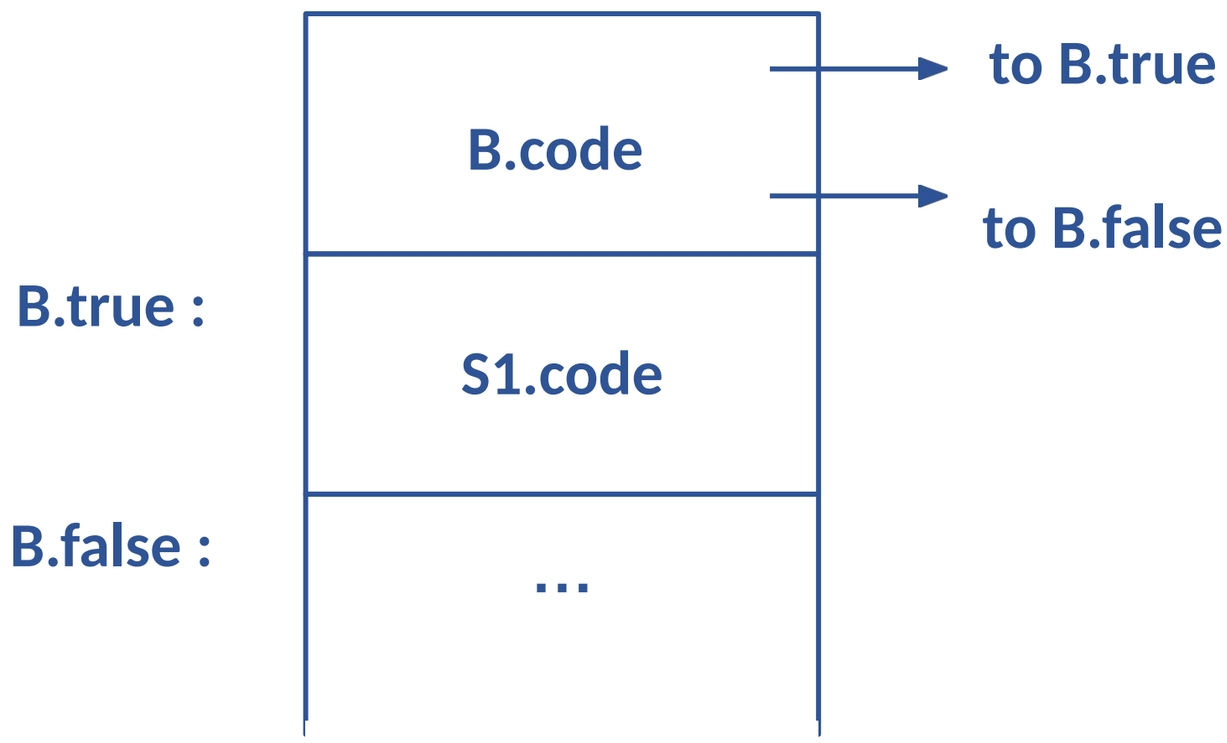
SDD to generate Intermediate Code - Conditions

Production	Semantic Rule
B -> id1 rel id2	*B.code = gen('if' id1.lexval rel.op id2.lexval 'goto' B.true)* *|| gen('goto' B.false)*
rel -> >	*rel.op = “>”*
rel -> <	*rel.op = “<”*
rel -> >=	*rel.op = “>=”*
rel -> <=	*rel.op = “<=”*

SDD to generate Intermediate Code - If Statement

Production	Semantic Rule
S -> if (B) S1	*{ B.true = new Label();* *B.false = S.next;* *S1.next = S.next;* *S.code = B.code || label(B.true) || S1.code; }*

SDD to generate Intermediate Code - If statement



SDD to generate Intermediate Code

- **If Else**

S -> if (C) S1 else S2

{C.true = new label();

C.false = new label();

S1.next = S.next;

S2.next = S.next;

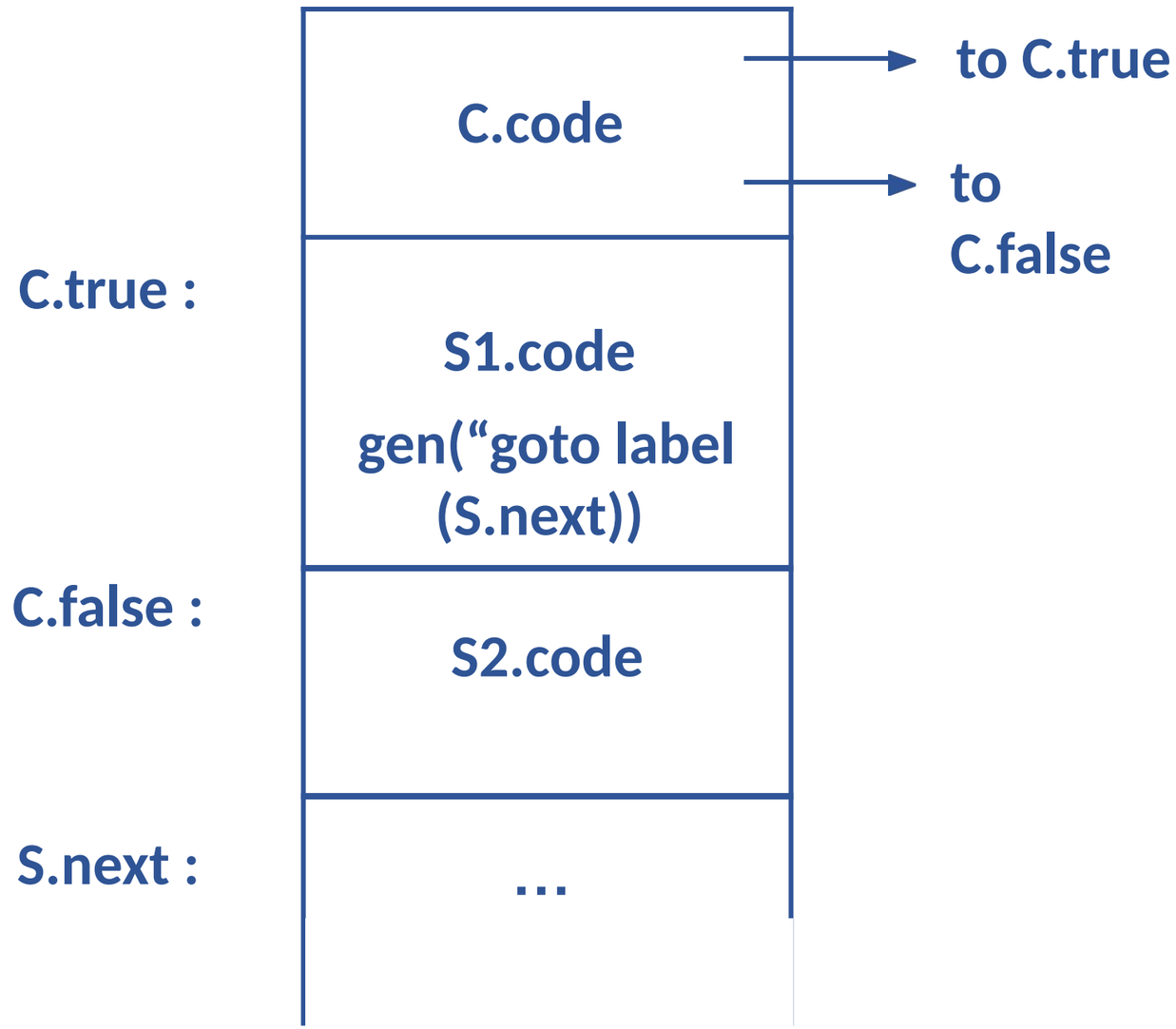
S.code = C.code || label(C.true) || S1.code || gen("goto" label(S.next))

|| label(C.false) || S2.code;}

SDD to generate Intermediate Code

- If Else

S -> if (C) S1 else S2



SDD to generate Intermediate Code

- **While**

S -> while (C) S1

{begin = new label();

C.true = new label();

C.false = S.next;

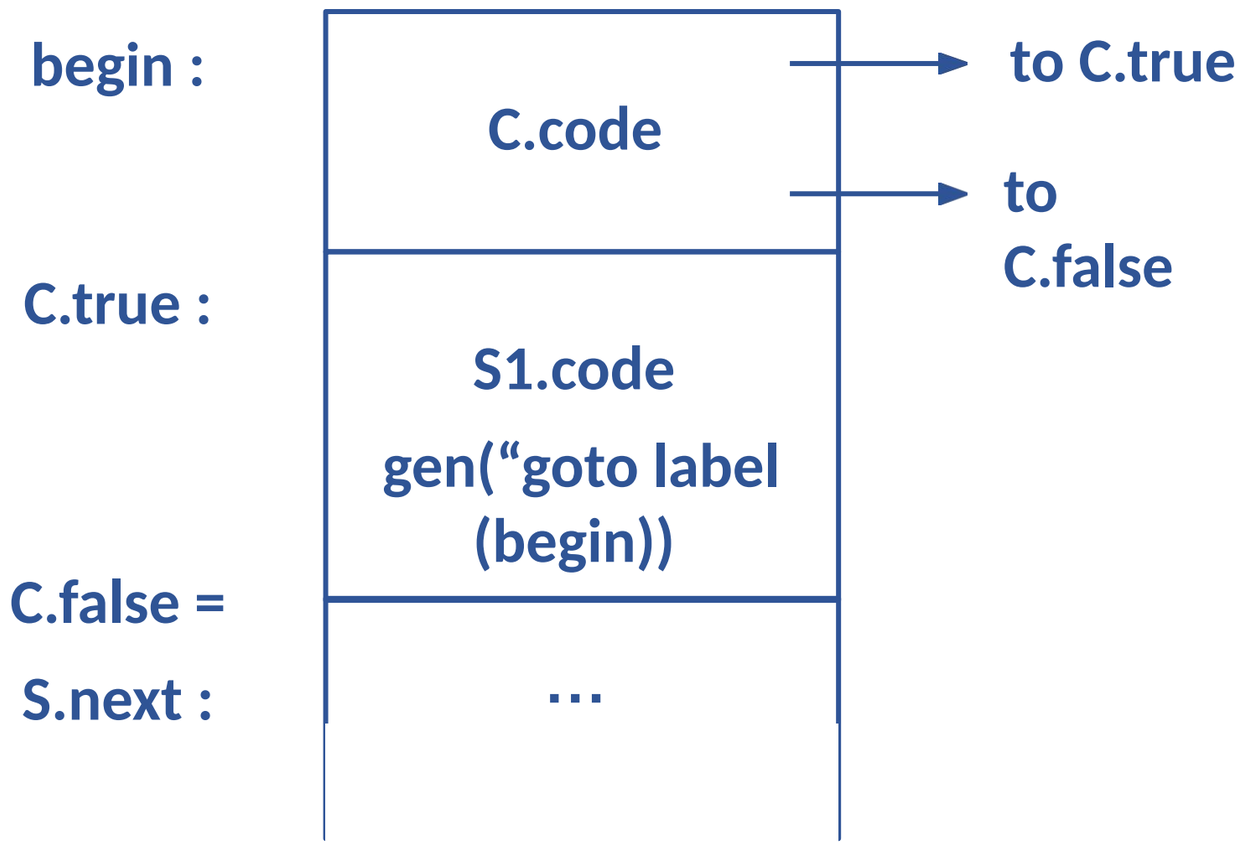
S1.next = begin;

S.code = label(begin) || C.code || label(C.true) || S1.code || gen("goto" label(begin));}

SDD to generate Intermediate Code

- While

S -> while (C) S1



SDD to generate Intermediate Code

- **Do - While**

S -> do (S1) while (C)

{C.true = new label();

C.false = S.next;

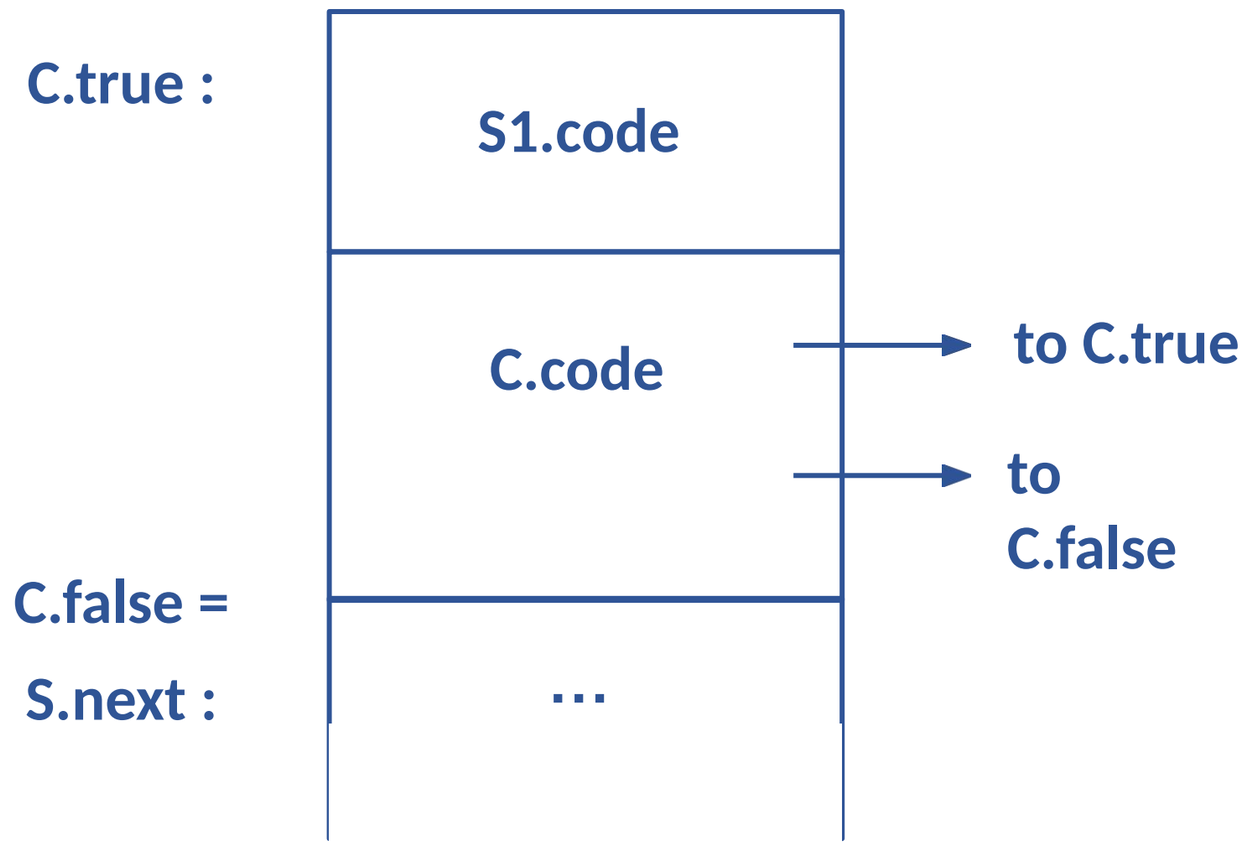
S1.next = NULL;

S.code = label(C.true) || S1.code ||C.code; }

SDD to generate Intermediate Code

- Do - While

S -> do (S1) while (C)



SDD to generate Intermediate Code

- **For**

S -> for (S1 ; C; S2) S3

{C.true = new label();

C.false = S.next;

S1.next = new label();

S2.next = S1.next;

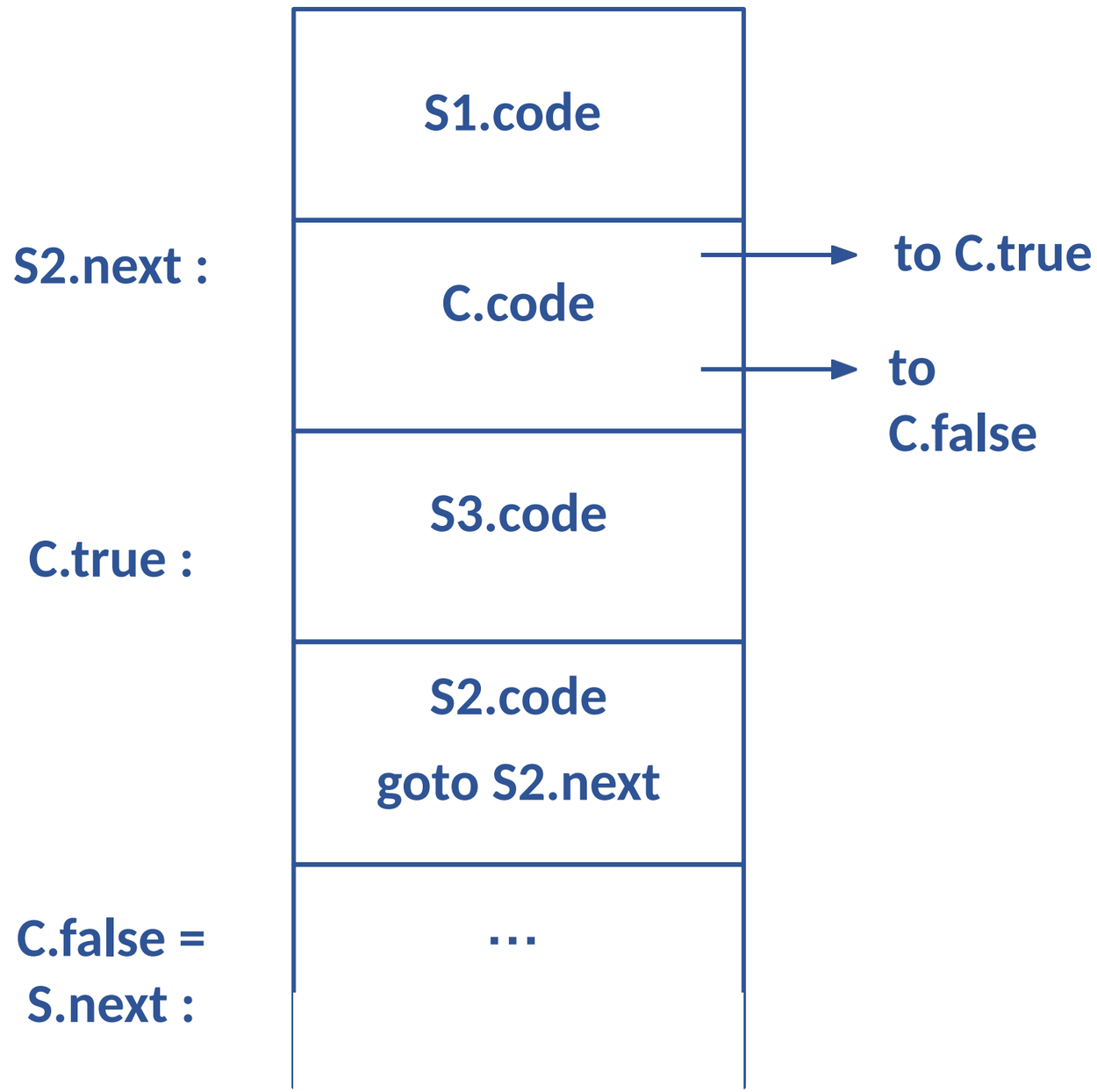
S3.next = NULL ;

S.code = S1.code || label(S1.next) || C.code || label(C.true) || S3.code || S2.code || gen("goto S1.next); }

SDD to generate Intermediate Code

- For

S -> for (S1 ; C; S2) S3



SDD to generate Intermediate Code

- **Boolean Expressions**

B -> B1 || B2

{ B1.true = B.true ;

B1.false = new label();

B2.true = B.true;

B2.false = B.false;

B.code = B1.code || label (B1.false) || B2.code; }

SDD to generate Intermediate Code

- **Boolean Expressions**

B -> B1 && B2

{ B1.false = B.false ;

B1.true = new label();

B2.true = B.true ;

B2.false = B.false ;

B.code = B1.code || label (B1.true) || B2.code; }

SDD to generate Intermediate Code

- **Boolean Expressions**

B -> ! B1 { *B1.true = B.false; B1.false = B.true; B.code = B1.code;* }

B → true { B.code = gen("goto" B.true);}

B → false { B.code = gen("goto" B.false);}

SDD to generate Intermediate Code

- **Generate Intermediate Code for the following example :**

```
if ( x > 10)
{
    x = x + 1
}
```

- **Grammar -**

S -> if (B) { S1 }

B -> id1 > id2

S1 -> id = E;

E -> E1 + E2

E2 -> id

THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

Compiler Design

Preet Kanwal
Department of Computer Science & Engineering

Teaching Assistant : Sree Pranavi G

Compiler Design

Unit 3: Syntax Directed Translation

Preet Kanwal

Department of Computer Science & Engineering

Lecture Overview

In this lecture, you will learn about -

- Syntax Directed Translation
- Design of Translation Schemes
- Types of Translation Schemes
- Problematic SDT
- Postfix schemes

Recap

- The Principle of Syntax Directed Translation states that the meaning of an input sentence is related to its syntactic structure, i.e., to its Parse-Tree.
- Translations for programming language constructs guided by context-free grammars.
- There are 2 kinds of attributes - Synthesized and Inherited.
- An SDD with only synthesized attributes is an S-attributed definition.
- Every S-attributed SDD is also L-attributed.

Recap - Syntax Directed Translation

- We associate **Attributes** to the grammar symbols representing the language constructs.
- Values for attributes are computed by **Semantic Rules** associated with grammar productions.
- There are two notations for attaching semantic rules:
  - **Syntax Directed Definitions** : High-level specification hiding many implementation details.
  - **Translation Schemes:** More implementation oriented, indicate the order in which semantic rules are to be evaluated.

Syntax Directed Translation Schemes

- **Translation Schemes** are more implementation oriented than syntax directed definitions since they indicate the order in which semantic rules and attributes are to be evaluated.
- A Translation Scheme is a context-free grammar in which,
  - Attributes are associated with grammar symbols.
  - Semantic Actions are enclosed between braces {} and are inserted within the right-hand side of productions.
- Yacc uses Translation Schemes.

Design of Translation Schemes

- When designing a Translation Scheme we must be sure that an attribute value is available when a semantic action is executed.
- When the semantic action involves only synthesized attributes, the action can be put at the end of the production.
- If wehave an L-Attributed SDD wemust enforce the following restrictions:
  - An inherited attribute for a symbol in the right-hand side of a production must be computed in an action before the symbol
  - A synthesized attribute for the non terminal on the left-hand side can only be computed when all the attributes it references have been computed - The action is usually put at the end of the production.

Problematic SDT

Implementation

- Ignoring the actions, parse the input and produce a parse tree as a result.
- Then, examine each interior node N, say one for production A -> α. Add additional children to N for the actions in α, so the children of N from left to right have exactly the symbols and the actions of α.
- Perform a preorder traversal of the tree, and as soon as a node labelled by an action is visited, perform that action.

Problematic SDT

Infix to prefix example

L -> En

E -> {printf(“+”);} E + T

E -> T

T -> {printf(“*”);} T * F

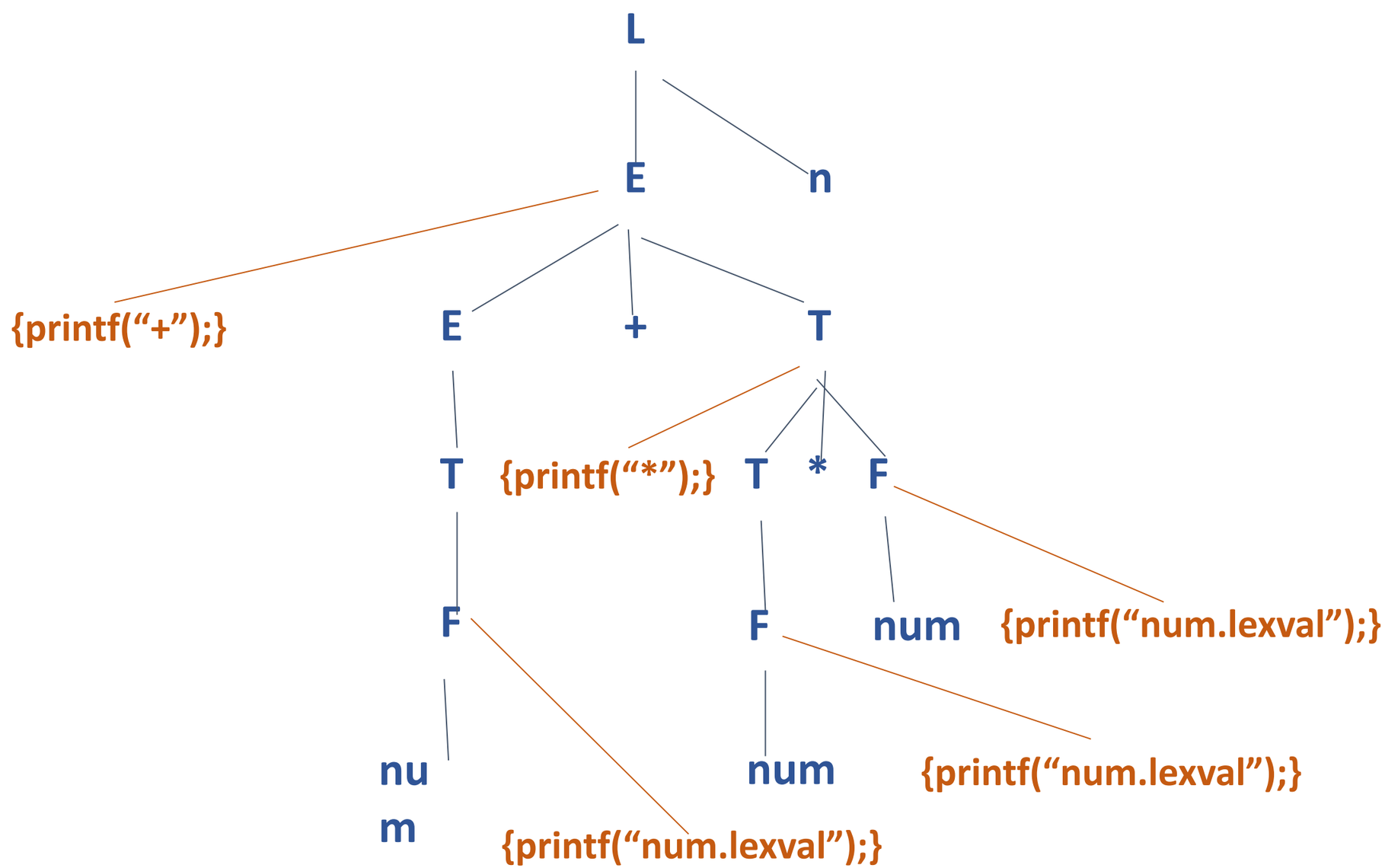
T -> F

F -> (E)

F -> num {printf(“num.lexval”);}

Problematic SDT

Infix to prefix example (contd.)



Problematic SDT

What does the following SDT scheme print for 5 + 4 - 2

E -> TR

R→+T {print("+");} R1

R→−T {print("-");} R1

R→ λ

T -> F

F -> num {print(num.lexval);}

Postfix SDT

- S attributed to SDT.

Evaluation of S-attributed SDD

- S-attributed SDDs will have only synthesized attributes and can be evaluated by a bottom up parser.

- Since the attributes in the semantic actions are only synthesized, the actions can be placed at the end of the production.

Postfix SDT

Rules for evaluation

- Consider the following production,

  A -> BCD

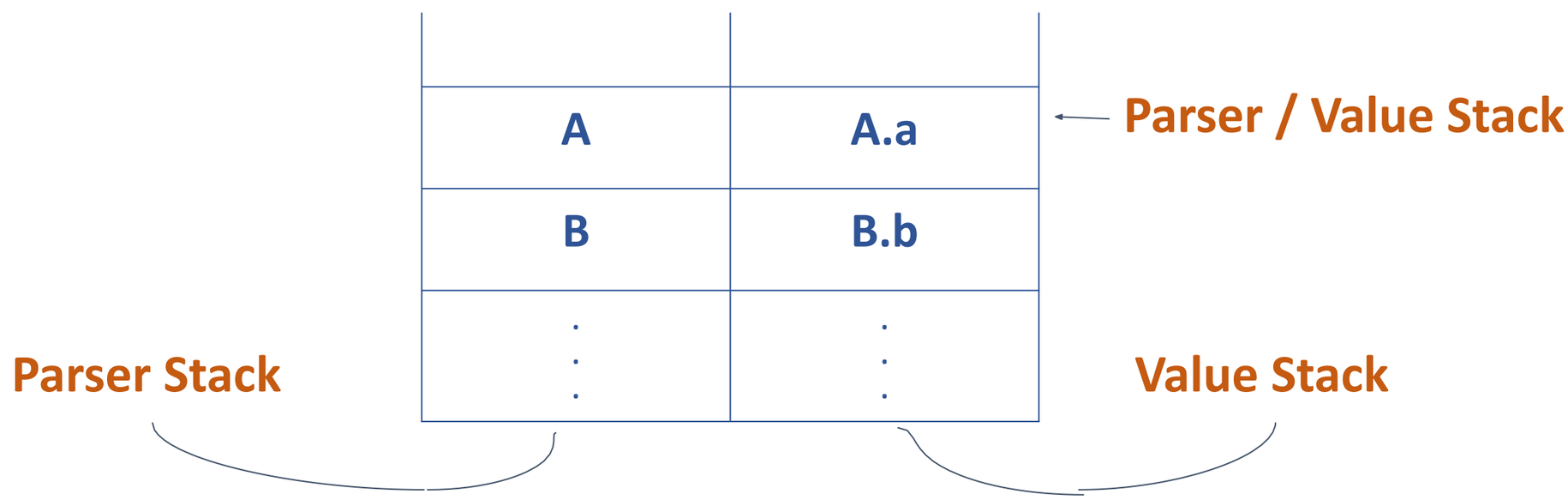
  before reducing BCD to A, the attributes of B, C and D must be computed before attribute of A which appears on the stack.

- Corresponding semantic action associated with the production must be executed.

Postfix SDT

Rules for evaluation

- The parser stack is extended to have parallel value stack.
- If the Action appears at the end of production in a SDT, such SDTs are called Postfix SDTs.



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

Compiler Design

Preet Kanwal
Department of Computer Science & Engineering

Teaching Assistant : Sree Pranavi G

Compiler Design

Unit 3: SDD to SDT

Preet Kanwal

Department of Computer Science & Engineering

Lecture Overview

In this lecture, you will learn about -

- L-attributed SDD to SDT
- SDT Implementation
- S-attributed SDD to SDT (Postfix SDT)

Conversion of L-attributed SDD to SDT

Translation by traversing a parse tree

- Build parse tree and annotate.
- Build the parse tree, add actions and execute the actions in pre-order.

Conversion of L-attributed SDD to SDT

Translation by traversing a parse tree

- Build parse tree and annotate.
- Build the parse tree, add actions and execute the actions in pre-order.

Conversion of L-attributed SDD to SDT

Translation during parsing

- Use a RDP with a function for each non-terminal.
- Generate code on the fly, using a RDP.
- Implement an SDT in conjunction with an LL parser.
- Implement an SDT in conjunction with an LR parser.

Conversion of L-attributed SDD to SDT

- Place the computation of inherited attributes for a non-terminal before that non-terminal appears in the right hand side of the production.

- Place the computation of synthesized attributes at the end of production.

Evaluation of L-attributed SDD

- L-attributed SDDs can have both synthesized attributes and inherited attributes.

Rule to be followed for evaluation

- Place the semantic rule corresponding to the inherited attributes of the non-terminal before the non-terminal appears on the right hand side of the production.

- Place the semantic rule corresponding to the synthesized attributes of the non-terminal at the end of the production.

Conversion of L-attributed SDD

Example

Production	Semantic Rule
T -> FT’	T’.ival = F.val; T.val = T’.val;
T ->*FT1’	T1’.ival = T.ival * F.val; T’.val = T1’.val;
T’ -> λ	T’.val = T’.ival;
F -> num	F.val = num.lexval

Conversion of L-attributed SDD

Example (contd.)

SDT

T -> F { T'.ival = F.val;} T' {T.val = T'.val;}

T' -> *F {T1'.ival = T'.ival * F.val;} T1' {T'.val =T1'.val;}

T' -> λ { T'.val = T'.ival;}

F -> num{ F.val = num.lexval; }

L-attributed SDD to SDT - Type Declarations

Production	Semantic Rule	Translation Scheme
D → T L	*{ L.in = T.type;}*	D → T { L.in := T.type } L
T → int	*{ T.type = integer; }*	T → int { T.type := 'integer' }
T → real	*{ T.type = float; }*	T → real { T.type := 'real' }
L → L_1 , id	*{ L_1.in= L.in; addType(id.entry, L.in); }*	L → { L1.in := L.in } L1, id { addtype(id.entry, L.in) }
L → id	*{ addType(id.entry, L.in); }*	L → id { addtype(id.entry, L.in) }

L-attributed SDD to SDT - Intermediate Code Generation

Convert the L-attributed SDD to generate intermediate code for If statement to SDT -

Production	Semantic Rule
S → if(C) S1	*C.true = new label();* *C.false = S.next;* *S1.next =* *S.next;* *S.code = C.code || label(C.true) || S1.code*

L-attributed SDD to SDT

Translation Scheme -

$S \rightarrow$ if (C) S1

$$\left\{ \begin{array}{l} C.true = new\ label\ (); \\ C.false = S.next \\ S1.next = S.next \end{array} \right\}$$

$$\left\{ \begin{array}{l} S.code = C.code\ || \\ label(C.true)\ || \\ S1.code \end{array} \right\}$$

L-attributed SDD to SDT

Convert the L-attributed SDD to generate intermediate code for while statement to SDT :

Production	Semantic Rule
S → while (C) S1	*begin = new label();* *C.true = new label();* *C.false = S.next* *S1.next = begin* *S.code = label(begin) || C.code || label(B.true) ||* *S1.code || gen('goto' label(begin));*

L-attributed SDD to SDT

Translation Scheme -

S → while (C) S1

begin = new label ();
C.true = new label ();

C.false = S.next;

S1.next = begin;

S.code = label(begin)
|| C.code ||
label(C.true) ||
S1.code || gen('goto'
label(begin));

SDT Implementation

There are two ways to implement a SDT scheme :

1. After Parsing - Work with the output of Parser (Parse Tree)
2. During Parsing

SDT Implementation - After Parsing

- Ignoring the actions, parse the input and produce a parse tree as a result.
- Examine each interior node N.
- Add additional children to N for the actions.
- Perform a preorder traversal of the tree, and as soon as a node labelled by an action is visited, perform that action.

SDT Implementation - After Parsing

- Consider the following translation scheme.

  S → ER

  R → *E { print("*"); } R | ε

  E → F + E { print("+"); } | F

  F → (S) | id { print(id.value); }

Here id is a token that represents an integer and id.value represents the corresponding integer value.

For an input 2 * 3 + 4, this translation scheme prints _____.

SDT - Infix to Postfix

Production	Semantic Rule
E → E1 + T	
E → T	
T → T1 * F	
T → F	
F → num	
F → id	

SDT - Infix to Postfix

Production	Semantic Rule
E → E1 + T	*E → {printf("+");} E1+ T*
E → T	*E → T*
T → T1 * F	*T → {printf("*");} T1* F*
T → F	*T → F*
F → num	*F → num {printf("%d", num.lexval);}*
F → id	*F → id {printf("%d", id.lexval);}*

Evaluation of S-Attributed Definitions

- Synthesized Attributes can be evaluated by a bottom-up parser as the input is being analyzed avoiding the construction of a dependency graph.
- The parser keeps the values of the synthesized attributes in its stack.
- Whenever a reduction $A \rightarrow \alpha$ is made, the attribute for A is computed from the attributes of α which appear on the stack.
- Thus, a translator for an S-Attributed Definition can be simply implemented by extending the stack of an LR-Parser.

Extending a Parser Stack

- Extra fields are added to the stack to hold the values of synthesized attributes.
- In the simple case of just one attribute per grammar symbol the stack has two fields - state and val.
- The current top of the stack is indicated by the pointer top.

Parser Stack

- Synthesized attributes are computed just before each reduction :
  - Before the reduction A → XYZ is made, the attribute for A is computed :

    A.a := f (s[top].val, s[top −1].val,
    s[top − 2].val).

state	val
Z	*Z.x*
Y	*Y.x*
X	*X.x*
…	…

Postfix SDT scheme for simple desk calculator

E → E1 +T {stack[top-2].val = stack[top-2].val + stack[top].val; top = top - 2; }

E → T

T → T1 * F {stack[top-2].val = stack[top-2].val * stack[top].val; top = top - 2;}

T → F

F → (E) {stack[top-2].val = stack[top-1].val; top = top - 2;}

F → digit

Implementing SDT Scheme during LR Parsing

- Introduce a Marker Non-terminal in place of each embedded action.

- There is one production for each Marker M,

  $M \rightarrow \lambda$

Example :

$A \rightarrow$ alpha {a} beta {b}

would convert to

$A \rightarrow$ alpha M beta {b}

$M \rightarrow \lambda$ {a}

Implementing SDT Scheme during LR Parsing

- For Example consider the following SDT scheme to generate Intermediate code :

P → {S.next = new label(); } S { P.code = S.code || label(S.next); }

S → id = num ; { S.code = gen(id.name "=" num.lexval); }

S → while ({ begin = new label (); C.true = new label (); C.false = S.next; } C) { S1.next = begin; } S1 { S.code = label(begin) || C.code || label(C.true) || S1.code || gen("goto" label(begin)); }

C → id_1 rel id_2 ; { C.addr = new Temp();
C.code = gen (C.addr "=" id1.name rel.op id2.name)
|| gen ("if" C.addr "goto" label(C.true)) || gen("goto" label(C.false)); }

Implementing SDT Scheme during LR Parsing

Introducing a Marker Non Terminal for every embedded action…

P → X S { P.code = S.code || label(S.next); }
X → λ {S.next = new label(); }

S → id = num ; { S.code = gen(id.name "=" num.lexval); }

S → while (M C) N S { S.code = label(begin) || C.code || label(C.true) || S1.code ||
gen("goto" label(begin)); }
M → λ {begin = new label (); C.true = new label (); C.false = S.next; }
N → λ {S1.next = begin;}

C → id_1 rel id_2 ; { C.addr = new Temp();
C.code = gen (C.addr "=" id1.name rel.op id2.name)
|| gen ("if" C.addr "goto" label(C.true))
|| gen("goto" label(C.false)); }

Implementing SDT scheme during LR Parsing

Parser Stack Structure

Stack record	*A*	*Synthesized attributes of A*
Record of Marker A	Inherited attributes of A	

Note : We perform general style of bottom-up parsing - shift-reduce parsing. A - non terminal

Implementing SDT scheme during LR Parsing

- Example continued…

Stack	Input Buffer	Action
$	while (a>b) a = 0; $	reduce using X → λ
$ X X, S.next = new label(); $	while (a>b) a = 0; $	Shift while X → λ { s[top].next = new label();}
$ X while while X, S.next = new label(); $	(a>b) a = 0; $	Shift (

Implementing SDT scheme during LR Parsing

- Example continued…

<table>
<tr><th>Stack</th><th>Input Buffer</th><th>Action</th></tr>
<tr><td>$ X while (
<table><tr><td>(</td></tr><tr><td>while</td></tr><tr><td>X, S.next = new label();</td></tr><tr><td>$</td></tr></table></td><td>a>b) a = 0; $</td><td>reduce using M → λ
M → λ { s[top].begin = new label();
s[top].true = new label();
s[top].false = s[top-3].next;}</td></tr>
<tr><td>$ X while (M
<table><tr><td>top</td><td>M, begin = new label();, C.true= new label();
C.false = S.next</td></tr><tr><td>-1</td><td>(</td></tr><tr><td>-2</td><td>while</td></tr><tr><td>-3</td><td>X, S.next = new label();</td></tr><tr><td></td><td>$</td></tr></table></td><td>a>b) a = 0; $</td><td>Shift id
(id.name = a)</td></tr>
</table>

Implementing SDT scheme during LR Parsing

- Example continued…

Stack	Input Buffer	Action
$ X while (M id id, id.name = a M, begin = new label(), C.true= new label(); C.false = S.next (while X, S.next = new label(); $	>b) a = 0; $	Shift rel (rel.op = >)

Implementing SDT scheme during LR Parsing

- Example continued…

<table>
<tr><th>Stack</th><th>Input Buffer</th><th>Action</th></tr>
<tr><td>$ X while (M id rel
<table><tr><td>rel , rel.op = “>”</td></tr><tr><td>id, id.name = a</td></tr><tr><td>M, begin = new label(); C.true= new label();
C.false = S.next</td></tr><tr><td>(</td></tr><tr><td>while</td></tr><tr><td>X, S.next = new label();</td></tr><tr><td>$</td></tr></table></td><td>b) a = 0; $</td><td>Shift id(id.name = b)</td></tr>
</table>

Implementing SDT scheme during LR Parsing

- Example continued…

Stack	Input Buffer	Action
$ X while (M id rel id	) a = 0; $	reduce using C → id rel id

top	id, id.name = b
-1	rel , rel.op = “>”
-2	id, id.name = a
	M, begin = new label(); C.true= new label(); C.false = S.next
	(
	while
	X, S.next = new label();
	$

```
C → id rel id { s[top-2].addr = new Temp();
s[top-2].code = gen(s[top].addr “=” s[top-2].name s[top-1].op s[top].name)
|| gen(“if” s[top].addr “goto” label(s[top-3].true)
|| gen(“goto” label(s[top-3].false);

top = top -2; }
```

Implementing SDT scheme during LR Parsing

- Example continued…

Stack	Input Buffer	Action
$ X while (M C C, C.addr = new Temp(); C.code M, begin = new label(); C.true= new label(); C.false = S.next (while X, S.next = new label(); $	) a = 0; $	Shift)

Implementing SDT scheme during LR Parsing

- Example continued…

<table>
<tr><th>Stack</th><th>Input Buffer</th><th>Action</th></tr>
<tr><td>$ X while (M C)
<table><tr><td></td><td>)</td></tr><tr><td></td><td>C, C.addr = new Temp(); C.code</td></tr><tr><td></td><td>M, begin = new label(); C.true= new label();
C.false = S.next</td></tr><tr><td></td><td>(</td></tr><tr><td></td><td>while</td></tr><tr><td></td><td>X, S.next = new label();</td></tr><tr><td></td><td>$</td></tr></table></td><td>a = 0; $</td><td>reduce using N → λ</td></tr>
</table>

Implementing SDT scheme during LR Parsing

- Example continued…

Stack	Input Buffer	Action
$ X while (M C)	a = 0; $	Shift id (id.name = a)

top	N, S.next = begin
-1	)
-2	C, C.addr = new Temp(); C.code
-3	M, begin = new label(); C.true= new label(); C.false = S.next
	(
	while
	X, S.next = new label();
	$

N → λ {s[top].next = s[top-3].begin;}

Implementing SDT scheme during LR Parsing

- Example continued…

<table>
<tr><th>Stack</th><th>Input Buffer</th><th>Action</th></tr>
<tr><td>$ X while (M C) N id
<table><tr><td>id, id.name = a</td></tr><tr><td>N, S.next = begin</td></tr><tr><td>)</td></tr><tr><td>C, C.addr = new Temp(); C.code</td></tr><tr><td>M, begin = new label(); C.true= new label(); C.false = S.next</td></tr><tr><td>(</td></tr><tr><td>while</td></tr><tr><td>X, S.next = new label();</td></tr><tr><td>$</td></tr></table></td><td>= 0; $</td><td>Shift =</td></tr>
</table>

Implementing SDT scheme during LR Parsing

- Example continued…

<table>
<tr><th>Stack</th><th>Input Buffer</th><th>Action</th></tr>
<tr><td>$ X while (M C) N id =
<table><tr><td>=</td></tr><tr><td>id, id.name = a</td></tr><tr><td>N, S.next = begin</td></tr><tr><td>)</td></tr><tr><td>C, C.addr = new Temp(); C.code</td></tr><tr><td>M, begin = new label(); C.true= new label();
C.false = S.next</td></tr><tr><td>(</td></tr><tr><td>while</td></tr><tr><td>X, S.next = new label();</td></tr><tr><td>$</td></tr></table></td><td>0; $</td><td>Shift num
(num.lexval = 0)</td></tr>
</table>

Implementing SDT scheme during LR Parsing

- Example continued…

<table>
<tr><th>Stack</th><th>Input Buffer</th><th>Action</th></tr>
<tr><td>$ X while (M C) N id = num
<table><tr><td>num, num.lexval = 0</td></tr><tr><td>=</td></tr><tr><td>id, id.name = a</td></tr><tr><td>N, S.next = begin</td></tr><tr><td>)</td></tr><tr><td>C, C.addr = new Temp(); C.code</td></tr><tr><td>M, begin = new label(); C.true= new label();
C.false = S.next</td></tr><tr><td>(</td></tr><tr><td>while</td></tr><tr><td>X, S.next = new label();</td></tr><tr><td>$</td></tr></table></td><td>; $</td><td>Shift ;</td></tr>
</table>

Implementing SDT scheme during LR Parsing

- Example continued…

<table>
<tr><th>Stack</th><th>Input Buffer</th><th>Action</th></tr>
<tr><td>$ X while (M C) N id = num ;

<table>
<tr><td>top</td><td>;</td></tr>
<tr><td>-1</td><td>num, num.lexval = 0</td></tr>
<tr><td>-2</td><td>=</td></tr>
<tr><td>-3</td><td>id, id.name = a</td></tr>
<tr><td></td><td>N, S.next = begin</td></tr>
<tr><td></td><td>)</td></tr>
<tr><td></td><td>C, C.addr = new Temp(); C.code</td></tr>
<tr><td></td><td>M, begin = new label(); C.true= new label();
C.false = S.next</td></tr>
<tr><td></td><td>(</td></tr>
<tr><td></td><td>while</td></tr>
<tr><td></td><td>X, S.next = new label();</td></tr>
<tr><td></td><td>$</td></tr>
</table></td><td>$</td><td>reduce using
S → id = num ;</td></tr>
</table>

S → id = num ; { s[top - 3].code = gen(s[top-3].name “=” s[top-1].lexval); }

Implementing SDT scheme during LR Parsing

- Example continued…

<table>
<tr><th>Stack</th><th>Input Buffer</th><th>Action</th></tr>
<tr><td>$ X while (M C) N S
<table><tr><td>top</td><td>S, S.code</td></tr><tr><td>-1</td><td>N, S.next = begin</td></tr><tr><td>-2</td><td>)</td></tr><tr><td>-3</td><td>C, C.addr = new Temp(); C.code</td></tr><tr><td>-4</td><td>M, begin = new label(); C.true= new label();
C.false = S.next</td></tr><tr><td>-5</td><td>(</td></tr><tr><td>-6</td><td>while</td></tr><tr><td></td><td>X, S.next = new label();</td></tr><tr><td></td><td>$</td></tr></table></td><td>$</td><td>reduce using
S → while(MC)NS</td></tr>
</table>

```
S → while(MC)NS {
s[top-6].code = label(s[top-4].begin ||
s[top-3].code || label(s[top-4].true || s[top].code ||
gen("goto" label(s[top-4].begin));

top = top - 6;
}
```

Implementing SDT scheme during LR Parsing

- Example continued…

<table>
<tr><th>Stack</th><th>Input Buffer</th><th>Action</th></tr>
<tr><td>$ X S
<table><tr><td>top</td><td>S, S.code</td></tr><tr><td>-1</td><td>X, S.next = new label();</td></tr><tr><td></td><td>$</td></tr></table></td><td>$</td><td>reduce using
P → X S</td></tr>
</table>

```
P → S {
s[top-1].code = s[top].code || label(s[top-1].next);

top = top - 1;
}
```

Implementing SDT scheme during LR Parsing

- Example continued…

Stack	Input Buffer	Action
$ P P, P.code $	$	Parsing Successful

Implementing SDT Scheme during LR Parsing

Semantic Actions written in terms of stack operations.

P → X S {s[top-1].code = s[top].code || label(s[top-1].next); top = top - 1; }
X → λ {s[top].next = new label();}

S → id = num ; {s[top - 3].code = gen(s[top-3].name "=" s[top-1].lexval);}
S → while (M C) N S { s[top-6].code = label(s[top-4].begin || s[top-3].code ||
label(s[top-4].true || s[top].code
|| gen("goto" label(s[top-4].begin)); top = top - 6; }
M → λ {s[top].begin = new label(); s[top].true = new label();
s[top].false = s[top-3].next;}
N → λ {s[top].next = s[top-3].begin;}

C → id_1 rel id_2 ; { s[top-2].addr = new Temp();
s[top-2].code = gen(s[top].addr "=" s[top-2].name s[top-1].op s[top].name)
|| gen("if" s[top].addr "goto" label(s[top-3].true)
|| gen("goto" label(s[top-3].false); top = top -2; }

Implementing SDT scheme during LR Parsing

Example 2 :

$S \rightarrow$ if ({ C.true = new label (); C.false = S.next; } C) { S1.next = S.next; } S1 { S.code = C.code || label(C.true) || S1.code }

Run over the input string if(x>y) x= 5; and provide all semantic actions in terms of stack operations.

Implementing SDT scheme during LR Parsing

Example 3 :

T -> F { T'.ival = F.val;} T' {T.val = T'.val;}

T' -> *F {T1'.ival = T'.ival * F.val;} T1' {T'.val =T1'.val;}

T' -> λ { T'.val = T'.ival;}

F -> num{ F.val = num.lexval; }

Run over the input string 3*5 and provide all semantic actions in terms of stack operations.

THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

Compiler Design

Preet Kanwal
Department of Computer Science & Engineering

Teaching Assistant : Kavya P K